

모든 저작권은 도서출판 홍릉 및
윤현주에게 귀속되어 있음.
복사 및 배포 금지

제 4 장

리눅스 들여다보기

목차

4. 리눅스 들여다보기.....	102
4.1 파일 시스템.....	102
4.1.1 파일 시스템 구조.....	103
4.1.2 파일의 속성.....	107
4.1.3 파일의 형식 (type).....	109
4.1.4 하드 디스크 구조.....	112
4.2 프로세스.....	116
4.2.1 프로세스 개념.....	116
4.2.2 실행 중인 프로세스 보기.....	118
4.3 사용자와 그룹.....	123
4.3.1 사용자 계정.....	123
4.3.2 그룹.....	124
4.3.3 사용자와 그룹 관리 기능.....	126
4.4 시스템 관리 기능.....	130
4.4.1 소프트웨어 관리.....	131
4.4.2 네트워크 설정과 관리.....	133
4장 마무리.....	142
보충 및 심화 학습.....	142

리눅스는 트리구조

리눅스엔 시시각각으로 변하는 프로세스와 시스템 상태를 파일처럼 나타내는 디렉토리 : /proc /sys

하나의 시스템 내에서 각 파일은 유일무이한 이름을 가진다

첫 번째 작업 위치는 홈, but not that name isnt home -> /home/id

파일의 속성 : 메타정보

각 파일에 대해 권한의 주체, 종류, 허용범위 정해짐

리눅스 파일 접근 권한은 소유자 그룹과 다른 사용자의 범주로 나뉜다 -> 모든 사용자 x

일반파일, 디렉터리 접근 권한 기능은 다르다 -> rwx이긴 한데 디렉터리에서의 의미는 다름

디렉토리 쓰기 권한 = 새 파일 생성 or 삭제 의미함

디렉토리 실행 권한 = 파일의 목록에 접근할 수 있음을 의미 = cd

리눅스 관리자 권한은 root 사용자만 가질 수 있는 것이 아니다 -> 특정 사용자에게 권한이 주어질 수 있다

리눅스 파일은 확장자를 반드시 붙이지 않아도 된다 but except .jpg

심볼릭 링크 = 윈도우의 바로가기와 같음

리눅스에선 모든 장치를 파일과 동일한 방식으로 취급함 = ex디바이스 하드웨어 상관없이 인풋 아웃풋 파일과 같
이 됨

블록형 장치 대표적 예 = 키보드가 아니다 -> 하드디스크이다 HDD

심볼릭 링크는 특수파일이다 -> 특수파일은 심볼릭링크, directory, socket, device

리눅스 -> 하나의 물리적 저장 장치를 여러 개의 파티션으로 나누어 사용 가능

리눅스에선 파티션 수가 많아도 하나의 트리 구조로 전체를 관리함

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

4. 리눅스 들여다보기

3장에서 GUI 환경의 리눅스 데스크톱을 활용해 사용 환경을 설정하고 기본 기능들을 사용해 보았다. 그런데 단순한 마우스 클릭과 메뉴 선택만으로 어떤 시스템 또는 운영체제의 기능을 충분히 활용하기는 부족하다. 1장에서 소개한 바와 같이 **운영체제의 기본 요소는 파일 관리, 프로세스(작업) 관리, 사용자 관리, 메모리 관리, 입출력 관리, 네트워크 관리, 보호 및 보안 기능** 등이다. 일반 사용자들이 운영체제의 이 모든 기능들을 사용하거나 이해할 필요는 없지만, 주로 “파일”에 대해 “작업”을 하고 여러 가지 응용 프로그램을 실행시키므로 최소한 파일 시스템의 구조와 프로세스라는 것에 대해 알아 둘 필요는 있다. 그리고 많은 사용자들에게 서비스를 제공하는 서버를 관리하는 기능을 익히는 것은 조금 차원이 다른 문제지만, 혼자 사용하는 경우에도 업데이트나 프로그램 설치 등 최소한의 시스템 관리는 해 주어야 한다.

4장에서는 리눅스의 내부로 조금 더 들어 가서 파일 시스템 및 프로세스 관리 구조를 살펴 보고, 사용자와 그룹, 소프트웨어 설치 등 시스템을 관리하는 여러 기능을 익힌다.

4.1 파일 시스템

사용자가 컴퓨터를 이용해서 하는 대부분의 작업이 파일시스템과 관련이 있다. 무엇이든 자료를 생성해서 저장하고 다시 재사용하기 위해서는 자료가 하드디스크나 USB 메모리 등의 저장장치에 기록이 되고, 이러한 **저장장치를 관리하는 운영체제의 요소가 파일 시스템**이다. 파일시스템은 수많은 **파일과 디렉터리들을 조직하여 관리**하고, **물리적 저장장치와 연관시켜 빠르고 안전하게 파일을 사용할 수 있도록 지원할 것**을 목적으로 구성되며 그 구조와 형식은 매우 다양하다. 예를 들어, Windows의 대표적인 파일시스템은 FAT32, NTFS 등이 있고, 유닉스/리눅스에서는 ext4, ReiserFS, JFS, XFS 등 매우 다양한 파일시스템이 있다. 리눅스 또는 기타 운영체제 설치 과정 중에는 파티션 설정, 파일시스템 지정 등의 단계가 반드시 필요한데, 이는 기본 저장 장치인 하드디스크를 어떤 방식으로 조직하고 관리하며 운영체제와 사용자를 지원할 것인가를 정하는 단계이기 때문이다. (물론 사용자의 편의를 위해 미리 정해진 대로 설치되는 경우도 있다.) 임베디드 시스템이나 모바일 기기와 같이 하드디스크가 아닌 저장 장치를 가진 경우도 그 특성에 맞는 중

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

류의 파일시스템을 반드시 설치해 주어야 운영체제와 시스템을 편리하게 사용할 수 있다.

각 파일시스템의 종류에 따른 특성이나 구조가 무엇인지, 실제로 물리적 장치가 어떻게 관리되는지는 컴퓨터공학의 운영체제 과목이나 기타 자료를 통해서 보다 깊이 있게 공부할 수 있고, 그에 따라 나에게 맞는 적절한 파일 시스템을 선택할 수 있다. 그러나 일반 사용자의 입장에서는 그런 내용을 알기도 어렵고 권장하는 대로 설치하여 사용하기 마련이다 (사실 알 필요도 없다!). 또한 운영체제 입장에서든 매번 변화하는 물리적 기기의 종류나 그를 관리하는 파일 시스템의 종류에 따라 사용자 지원 방식을 바꾸기 힘들기 때문에 앞 장에서 설명한 대로 여러 ‘계층’을 두어서 여러 기능을 지원하게 된다. 따라서 사용자가 보는 파일 시스템은 실질적인 파일시스템 위에 한 겹의 껍데기를 덧씌워 어떤 파일시스템이든 사용자에게 똑같이 보이도록 하는 ‘가상(virtual)’ 파일 시스템이며, 유닉스/리눅스의 종류나 버전이 무엇이든 하드웨어가 어떤 것이든 그를 관리하는 파일 시스템의 종류가 무엇이든 관계없이 항상 동일한 방식의 뷰와 서비스를 제공한다.

이번 장에서는 유닉스/리눅스에서 사용자가 직접 접하게 되는 가상 파일 시스템 구조를 살펴 보도록 한다. 참고로 모든 예는 Ubuntu 20.04 리눅스에서 실행되었는데, 리눅스의 종류에 따라 모양이나 기능이 약간씩 차이가 있겠지만 기본적으로 같은 구조에 겉모습만 다른 것임을 알고 가자. WSL로 우분투를 사용하는 경우 주어진 이름을 가진 프로그램을 설치한 후 실행시키면 된다.

4.1.1 파일 시스템 구조

물리적인 저장장치의 형태나 저장 방식과는 상관없이 파일시스템은 모든 자료를 ‘파일(file)’이라는 단위로 관리한다. 즉, 파일은 자료의 논리적인 저장 단위이다. 그리고 관련 있는 여러 파일들을 모아 ‘디렉터리(directory)’라는 논리적 집합 단위를 생성할 수 있다. Windows에서 ‘폴더(folder)’라 칭하는 것과 유사하다(리눅스에서도 GUI 환경에서는 폴더라 한다).

내부 구성이나 작동 방식과는 관계 없이 현재 대부분의 운영체제에서 사용자가 인식할 수 있는 파일시스템의 구조는 뒤집힌 트리(tree) 형태이다. 그림 4-1에서 보는 바와 같이 하나의 뿌리(루트, root)에서 시작하여 가지가 뻗어 나가듯 디렉터리와 파일들이 계층적으로 조직화된다. 이 구조에서 어떤 디렉터리의 아래에 있는 디렉터를 하위 디렉터리(subdirectory)라 하고, 그 반대는 상위 디렉터리 또는 부모 디렉터리라 한다.

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

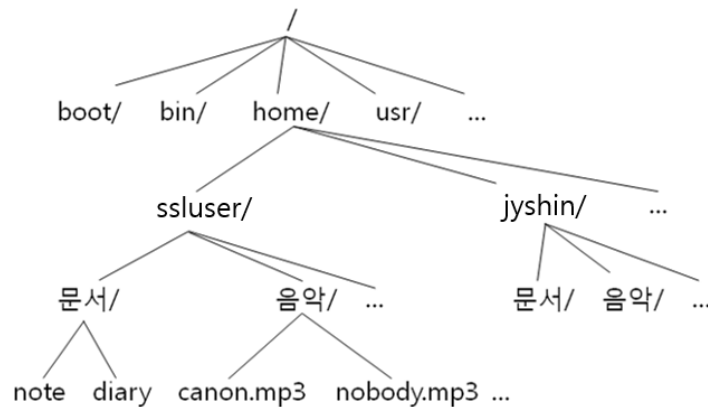


그림 4-1 파일시스템 구성 예

GUI 환경에서는 아이콘이나 메뉴를 클릭해서 폴더를 이동할 수 있는데, 시스템 내부에서는 이들을 관리하기 위해 특별한 표기법과 명칭을 사용한다. 그림 4-1에서 보는 것처럼 루트는 ‘/’로 표시된다. 일반적인 트리 구조에서와 마찬가지로 트리의 위쪽에 있는 디렉터리는 “부모”가 되고 아래쪽에 있는 파일이나 디렉터리는 “자식”이 된다. 그 외 위치와 관련된 특수 문자는 표 4-1에 나타나 있다. 그림 4-1에서 **각 이름 뒤에 붙은 ‘/’ 문자는 디렉터리임을 표시하는 역할을 한다.**

표 4-1 특정 위치를 표기하는 특수 문자



표기	위치
/	루트
~	홈 디렉터리
.	현재 디렉터리
..	부모 디렉터리

어떤 디렉터리나 파일의 위치는 계층 구조 상에서의 **경로(path)** 정보로 표현할 수 있다. **절대(absolute) 경로**는 루트에서 시작하여 계층 구조를 따라 내려 오면서 도달하는 길, **상대(relative) 경로**는 현재 위치를 기준으로 대상에 이르는 길을 명시한다. **파일이나 디렉터리의 이름은 전체 시스템에서 유일무이한 이름으로 명명되어야 하는데, 트리 구조의 특성 상 ‘절대 경로’가 이름의 역할을 하게 된다.** 즉, **절대 경로가 다르다면 같은 이름의 파일을 여러 군데 만들 수 있으며, 역으로 상대 경로로서 이름이 같아 하더라도 그것이 꼭 같은 파일을 의미하지는 않는다.** 그림 4-1에서 ssluser 디렉터리와 jyshin 디렉터리는

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

동일한 이름의 하위 디렉터리로 ‘음악’ 디렉터리를 가지고 있지만 절대 경로가 다르기 때문에 문제 없다. 어떤 한 위치에서 절대 경로와 상대 경로 또는 특수 문자를 이용해 각 파일을 지칭하는 방법을 살펴 보자.

```
현재 위치: /home/ssluser/문서
diary의 절대 경로: /home/ssluser/문서/diary
diary의 상대 경로: diary (./diary)
canon.mp3의 절대 경로: /home/ssluser/음악/canon.mp3
canon.mp3의 상대 경로: ../음악/canon.mp3
jyshin의 절대 경로: /home/jyshin 또는 ~jyshin
jyshin의 상대 경로: ../../jyshin
```

이런 구조를 머리에 두고 다시 GUI 환경의 파일 관리자(프로그램명: nautilus)를 실행해 보자. 그림 4-2의 파일 관리자의 왼쪽에 보이는 **홈**은 사용자의 홈 디렉터리를 의미하여 그림 4-1의 구조에서 `/home/ssluser` 위치에 존재한다. 홈의 위치는 관리자 설정에 의해 계정 생성 시 자동으로 정해지며, 우분투의 경우 홈 폴더를 쉽게 접근할 수 있도록 처음 설치 시 첫 사용자의 홈 폴더 바로가기를 바탕화면에 생성해 둔다.

파일 관리자에서 루트를 보려면 `/` 를 누르고 **Enter** 키를 누르거나, 좌측 위치 메뉴 중 **다른 위치**를 선택하여 **컴퓨터**를 눌러 이동하면 된다.

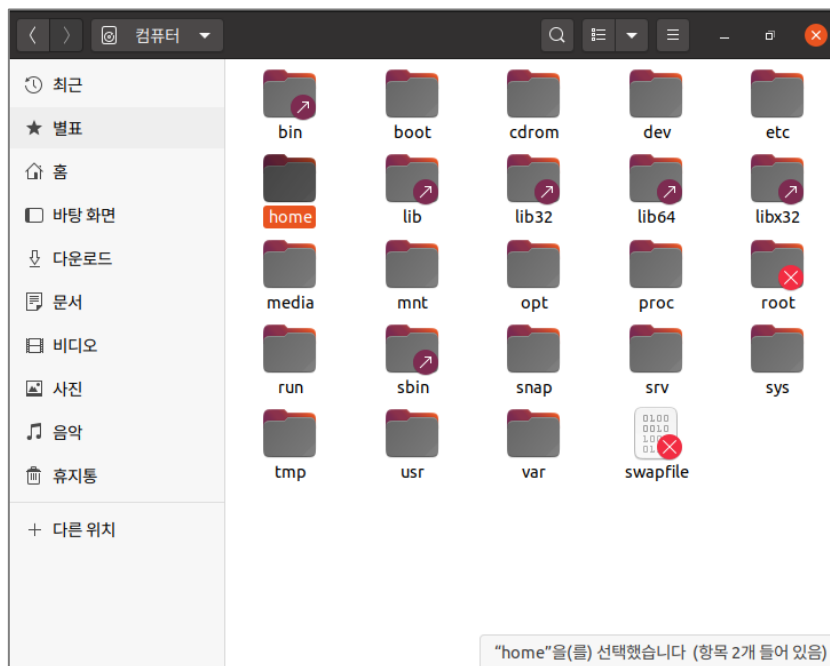


그림 4-2 파일 시스템의 루트 폴더

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

유닉스/리눅스에서는 특정 파일들을 특정 디렉터리에 저장하도록 파일시스템 계층 구조에 대한 관습과 표준이 정해져 있어, 시스템이 변경되거나 개선되어도 사용자가 특정 파일의 위치를 쉽게 예측할 수 있으며, 새로운 프로그램을 개발하거나 설치하고자 할 때 가이드라인 역할을 한다. 실제로 설치된 시스템을 확인해 보면 이 표준에 따라 설치 파일들과 프로그램 파일들이 저장되어 있는 것을 볼 수 있다. 표 4-2는 대표적인 디렉터리와 그 내용에 대한 표준을 설명한다. 실제 루트 디렉터리에는 운영체제에 따라 이 외에 더 많은 디렉터리가 있을 수 있다.

표 4-2 유닉스/리눅스 파일 계층 구조 표준

디렉터리	저장 파일
/boot	부팅에 필요한 모든 파일
/bin	핵심적인 사용자 명령 파일 (binary)
/dev	장치 파일. 하드 디스크, 터미널, 주변 장치 등과의 연결 역할
/etc	텍스트로 된 설정(configuration) 파일. 실행 가능한 파일은 올 수 없다.
/home	각 사용자별 저장 공간. 사용자 로그인 시 시작 위치가 된다.
/lib	핵심적인 공유 라이브러리와 커널 모듈
/media	Floppy, CD-ROM 드라이브 등 탈부착 가능한 장치 파일
/mnt	임시로 연결할 파일시스템에 대한 연결(mount) 지점
/opt	추가로 설치할 응용 소프트웨어 패키지
/root	시스템 관리자(root)의 홈 디렉터리
/sbin	시스템 관리에 사용되는 프로그램 파일.
/usr	읽기 전용의 공유 데이터 저장. 여러 시스템에 공통적으로 적용될 수 있는 것들만 저장하며 특정 호스트에 관련되거나 시간에 따라 변경되는 데이터는 저장하지 않아야 한다.
/usr/bin	거의 모든 사용자 명령 파일. /bin에 없는 프로그램도 많다.
/usr/include	C 프로그래밍에 사용되는 헤더 파일
/usr/lib	프로그래밍을 위한 라이브러리 및 패키지
/usr/local	시스템 관리자가 별도로 설치하는 소프트웨어.
/usr/src	리눅스 커널의 소스 코드. 설치의 선택 사항이며, 컴파일러가 설치되면 커널 헤더 파일들이 여기에 설치된다.
/var	시간에 따라 변하는 데이터 파일.
/proc	실행 중인 프로세스 및 시스템 상태에 대한 정보 저장. 가상 파일 시스템으로 실제로는 메모리에 위치하며 시스템 종료 시 삭제된다.
/sys	하드웨어 및 장치 드라이버, 커널 하위 시스템 등에 대한 정보를 제공하는 가상 파일 시스템. /proc과 마찬가지로 시스템 운영 중에만 존재함.
/tmp	임시 파일용 디렉터리. 시스템과 사용자 관계 없이 임시 파일들을 저장할 수 있는 공간이며 여기에 저장한 파일은 예고 없이 삭제될 수 있다.

모든 저작권은 도서출판 흥릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

4.1.2 파일의 속성

파일은 기본적으로 자료를 저장하는 단위이다. 그런데 수없이 많은 파일들을 관리하기 위해서는 자료 내용뿐만 아니라 파일 그 자체에 대한 정보, 즉 메타정보(metadata)도 관리해야 할 필요가 있다. 메타정보에는 파일의 고유번호, 이름, 형식, 소유자, 생성 날짜, 변경 날짜, 크기, 접근 권한, 파일 시스템 구조에서의 위치, 현재 상태 등 많은 속성이 포함된다. 어떤 속성을 저장하고 관리할 것인지는 운영체제 및 운영체제에 속한 파일시스템이 결정하게 된다.

그림 4-3은 리눅스 데스크톱에서 파일의 속성을 본 결과다. 파일 관리자에서 보고 싶은 폴더 또는 파일에 마우스를 올려 놓은 후 오른쪽 버튼을 클릭한 다음 속성 메뉴를 선택하면, 그림과 같이 속성을 보여 주는 창이 뜬다. 기본 속성 외에도 각각의 탭을 선택하여 상세한 속성을 볼 수 있다.



그림 4-3 파일의 속성 보기

그림 4-3에서는 두 가지 파일 형식(type)을 볼 수 있다. 왼쪽은 보통 텍스트 문서이고 오른쪽은 폴더(디렉터리)이다. 시스템에서 구분하는 파일의 종류 중에 일반 사용자가 가장 흔하게 접하는 것이 바로 일반 파일과 디렉터리이다.

일반 파일은 다시 그 양식에 따라 매우 많은 종류가 있다. MS Windows에서 각 파일의 확장자(extension)를 이용해 파일의 형식을 구분하는 것과 달리 유닉스/리눅스에서는 확장자가 정해지지 않고 파일의 앞부분에 형식에 대한 정보를 저장하는 방식으로 파일을 만들기도 한다. 즉, 텍스트 파일의 경우 Windows에서는 .txt라는 확장자가 사용되는 것이 일반적이지만 유닉스/리눅스에서는 확장자와 관계 없이 내용이 일반 텍스트로 구성된 경

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

우 텍스트 파일로 인정한다. 마찬가지로 실행 가능한 파일의 경우 .exe 확장자를 사용해 하는 Windows와 달리, 파일의 이름과 상관 없이 맨 앞 부분(file header)에 미리 약속된 형태의 정보가 저장되어 있는 ELF(Executable and Linkable Format) 양식을 따르면 실행 가능한 이진(binary) 파일로 인정된다. 텍스트 파일 중에서도 약속된 명령어로 구성된 파일은 실행 가능한 것으로 속성을 지정하여 사용할 수 있다.

텍스트와 실행 파일 외 특정 응용 프로그램에 의해 생성되거나 처리되는 일반 파일의 경우 정해진 확장자를 따라야 한다. 이미지를 저장하는 .jpg 또는 .png, 영상을 저장하는 .mp4, 리브레오피스에 의해 생성되는 .odt, odp 등이 그 예라고 할 수 있다.

결들임: 파일에는 실제로 무엇이 저장되나요?

컴퓨터에 저장되는 모든 정보는 사실 똑같은 형태로 저장된다. 즉, 0과 1로 이루어진 이진 데이터이다. 그러면 어떤 이진수는 문자로 해석하고, 어떤 이진수는 그림으로 해석하며, 어떤 이진수는 프로그램 실행 명령으로 인식하는가? 이는 전적으로 그 파일을 해석하는 프로그램에 달린 문제다. 그리고 파일에 저장되는 내용 중에는 이 파일을 어떤 식으로 해석해야 한다는 제어 정보가 포함된 경우가 많다. 각 프로그램은 파일의 내용을 읽어서 약속된 논리 구조에 맞는지 점검하고 그 구조에 따라 해석하여 문자를 출력하기도 하고 그림을 보여 주기도 하며, 직접 실행을 하기도 한다. 사용자가 처음 접하게 되는 운영체제 인터페이스인 셸(shell)의 첫 번째 역할도 사용자 명령의 해석과 파일 내용의 해석이다.

속성 중에서 **권한** 속성을 보도록 하자. MS Windows와 달리 유닉스/리눅스는 다중 사용자를 지원하는 것이 기본이기 때문에 각 파일이나 디렉터리에 대해서도 다양하고 엄격한 접근 권한(access right)을 설정하도록 할 수 있다. 권한은 소유자, 소유자의 그룹, 모든 사용자(기타) 등 3가지 부류에 대해 각각 다르게 설정할 수 있다. 그림 4-4는 일반 파일과 폴더에 대한 권한을 각각 보여 준다. 일반 파일은 읽기(r), 쓰기(w), 실행(x)에 대해 권한을 설정할 수 있으며, 읽기 전용으로 설정하거나 권한을 아예 불허하는 것도 가능하다. 디렉터리에 대한 권한은 파일의 목록을 보거나(r) 디렉터리 내부에 들어가서 파일에 접근하거나(x) 파일을 생성/삭제할 수 있는(w) 권한으로 구성된다. 내부 파일의 권한 바꾸기를 선택하면 디렉터리 내에 포함되어 있는 파일들의 권한을 한꺼번에 변경할 수 있다.

모든 저작권은 도서출판 홍릉 및
윤현주에게 귀속되어 있음.
복사 및 배포 금지

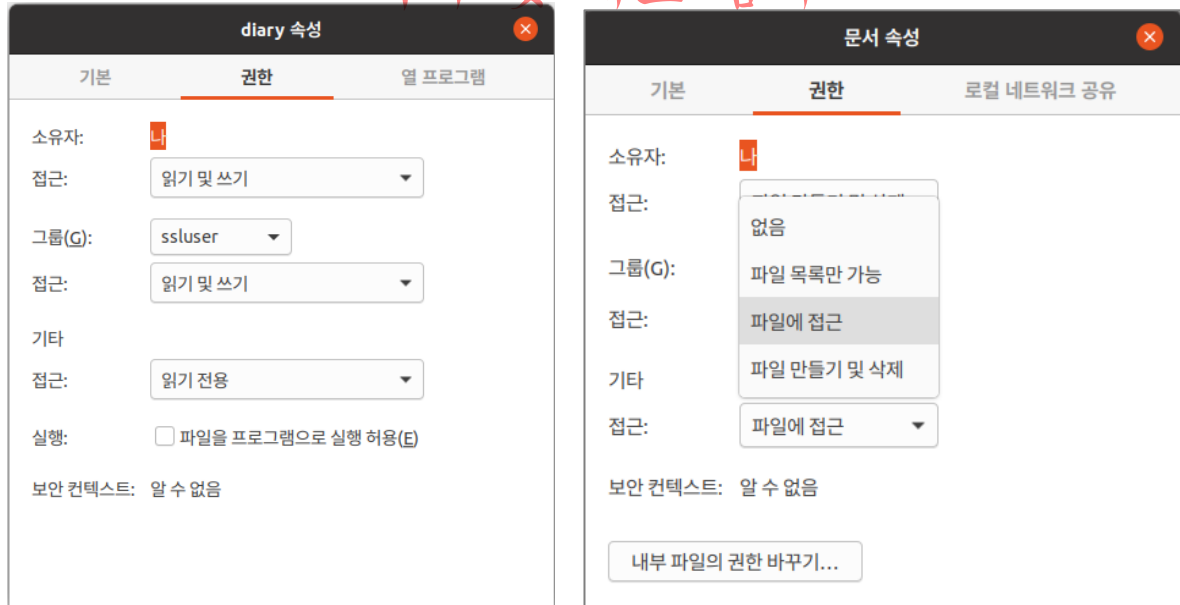


그림 4-4 접근 권한 속성

자신의 파일을 다른 사람들이 읽거나 변경하지 못하도록 하려면 각 파일에 대해 권한을 **없음** 또는 **읽기 전용**으로 설정한다. 디렉터리에 대해서도 마찬가지로 설정할 수 있다. 단, 각 경우에 대해 아래쪽의 권한은 위쪽의 권한을 포함한다. 즉, **파일 만들기 및 삭제**를 허용하면 **파일 목록 읽기**나 **파일 접근**이 가능하다. GUI가 아닌 CLI 환경에서 명령어로 권한을 설정할 때는 각 권한을 분리해서 좀 더 세밀하게 설정할 수 있다.

4.1.3 파일의 형식 (type)

유닉스/리눅스 운영체제의 특징 중 하나는 자료를 저장하는 **일반적인 파일 외에** 이를 조직화한 디렉터리 및 주변 장치와 프로세스 등 시스템 실행과 **관련된 모든 것을 파일로 취급한다는 것이다**. CD-ROM 드라이브, USB 메모리, 네트워크를 통한 사용자 연결, 프로세스 간에 메시지를 주고 받기 위한 소켓(socket)이나 파이프(pipe), 프린터와 통신하기 위한 포트, 현재 실행 중인 프로세스 등 **모든 것이 파일로 표현되어 파일 시스템 계층 구조 안에 포함된다**. 이렇게 **파일이 아닌 것을 파일처럼 취급하는 것은 사용자 또는 프로그래머가 시스템의 다양한 자원을 파일을 다루듯이 쉽게 사용할 수 있도록 하기 위함이며, 이런 파일들을 특수(special) 파일이라고 한다**. 특수 파일은 표 4-3에 보는 바와 같이 분류할 수 있다.

모든 저작권은 도서출판 홍릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

표 4-3 유닉스/리눅스의 특수 파일

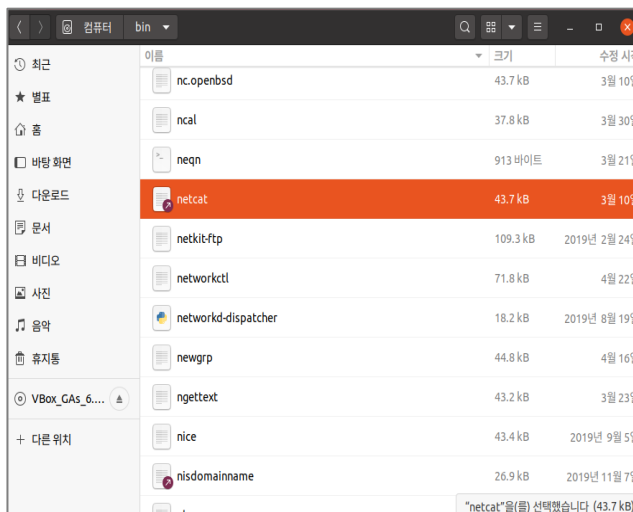
파일 종류	대상 및 특징
디렉터리 (directory)	여러 파일의 논리적 집합 단위. 계층 구조를 형성한다.
심볼릭 링크 (symbolic link)	다른 파일에 대한 참조 표시. 실제로는 원래 파일을 찾아갈 수 있는 경로 정보를 저장한다. (Windows의 바로 가기와 유사)
장치 (device)	하드웨어 장치. 순차적 방식으로 입출력하는 문자(character)형 장치와 임의 접근이 가능한 블록(block)형 장치로 구분된다.
기명 파이프 (named pipe)	실행 중인 프로세스 간에 정보를 전달하기 위한 통로. FIFO 형태로 운영되어 한 프로세스의 출력이 다른 프로세스의 입력으로 전달된다.
소켓 (socket)	프로세스 간 정보 교환을 위한 통로. 파이프와 달리 양방향 정보 전달이 이루어진다.

디렉터리는 파일의 고유 번호와 이름을 저장하고 있는 특수한 파일로서 그 내용은 시스템에 의해서만 작성되고 변경된다. 즉, 각 사용자는 새로운 디렉터리를 생성하거나 디렉터리 내에 파일을 생성하거나 삭제하고, 디렉터리 자체를 삭제하는 등의 작업을 수행할 수 있지만, 어떤 디렉터리가 어떤 파일들로 구성되어 있으며 부모는 누구인가 하는 등의 디렉터리 내부 정보를 직접 건드릴 수는 없다. 시스템은 디렉터리라는 특별한 파일의 내용을 기록하고 관리하여 앞에서 언급한 트리 모양의 파일시스템 구조를 관리할 수 있으며, 실제로 하드디스크에 저장된 파일들에 접근하기 위한 안내 역할을 한다.

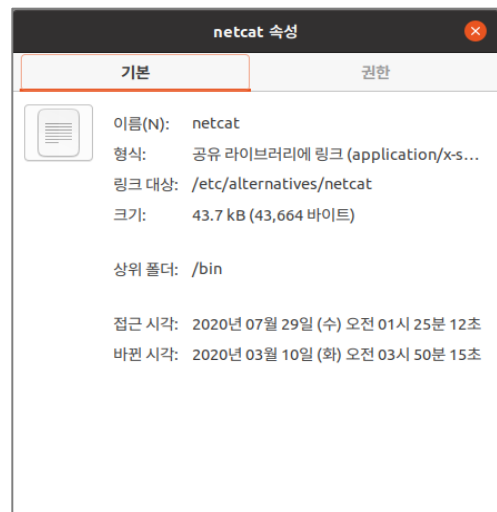
심볼릭 링크는 다른 곳에 존재하는 파일이나 디렉터리를 찾아갈 수 있도록 경로 정보를 저장하고 있는 특수한 파일이다. 사용자는 링크 생성 명령을 사용해 링크를 만들고, GUI 환경에서 링크 파일을 더블 클릭하거나 명령어를 사용해서 실제 파일을 접근할 수 있다. 앞에서 본 트리 구조를 따르자면 어떤 하나의 파일에 접근하기 위해 여러 번의 더블 클릭 또는 긴 경로 이름을 사용해야 하는 번거로움이 있을 수 있는데, 이를 짧게 줄여 주는 장점이 있다. 또한 연관 있는 파일들을 하나의 디렉터리에 모으고 싶는데 한 파일이 여러 디렉터리에 관련될 경우 복사본을 여러 개 둔다면 관리하기가 매우 어렵다. 이 때, 원본 파일은 한 군데에만 저장하고 다른 디렉터리에는 그 곳에 접근할 수 있는 경로 정보만을 저장한다면 관련 정보를 한 곳에 모으는 것과 파일 관리의 일관성 유지가 동시에 가능하게 된다. 링크에는 심볼릭 링크(soft link라고도 함) 외에 **하드 링크(hard link)**라는 방식도 있는데, 이는 실제 파일에 이름을 추가로 지정해 주는 것으로서 특수 파일이 아니라 일반 파일로 취급된다. 두 링크의 내부 구조와 차이점 등 상세한 사항은 추후 관련 명령어와 프로그래밍을 다룰 때 다시 살펴 보도록 하겠다.

모든 저작권은 도서출판 흥릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

그림 4-5는 심볼릭 링크의 속성이 어떻게 나타나는지 보여 준다. /bin 디렉터리는 사용자가 실행할 수 있는 명령, 즉 프로그램들을 모아 놓은 곳인데, 살펴 보면 여러 개의 심볼릭 링크가 있음을 알 수 있다. 아이콘에 화살표가 덧붙여져 있는 것이 링크이며, 이 중 netcat라는 이름의 파일을 선택해 속성을 열어 보면 그림 4-5(b)와 같다. 파일의 형식은 “실행파일에 링크”로 되어 있고 링크 대상은 “/etc/alternatives/netcat”이다. 즉, /bin/netcat라는 파일은 실체가 아니며, 여기에 설정된 경로를 따라서 원본 파일에 접근할 수 있다는 것이다. netcat라는 프로그램은 특성상 /etc 디렉터리에 소속되는 것으로 분류되었지만, 사용자가 실행할 수 있는 프로그램이기 때문에 /bin을 통해서도 접근할 수 있도록 링크를 설정한 것이다.



(a) /bin 디렉터리



(b) /bin/netcat 파일의 속성

그림 4-5 심볼릭 링크 속성

특수 파일들은 파일 시스템 내 어디나 존재할 수 있지만 **장치 파일의 경우는 대개 /dev와 같이 약속된 장소에 저장한다**. 그림 4-6은 /dev 폴더의 내용과 그 중 sda 파일의 속성을 보여 준다. 임의 접근이 가능한 블록형 장치인데 특이한 것은 파일의 크기가 0이라는 것이다. **장치 파일은** 우리가 보통 사용하는 텍스트나 바이너리 데이터를 포함하는 것이 아니라 **실제 하드웨어 장치를 사용하는 통로로서 인터페이스 역할을 하는 것이기 때문에, 저장된 데이터의 크기를 나타내는 크기 정보는 0이 되는 것이다**. 더 상세한 내용은 본 서의 범위를 벗어나므로 각 개인의 열정에 맡기도록 하겠다.

모든 저작권은 도서출판 흥릉 및
윤현주에게 귀속되어 있음.
복사 및 배포 금지

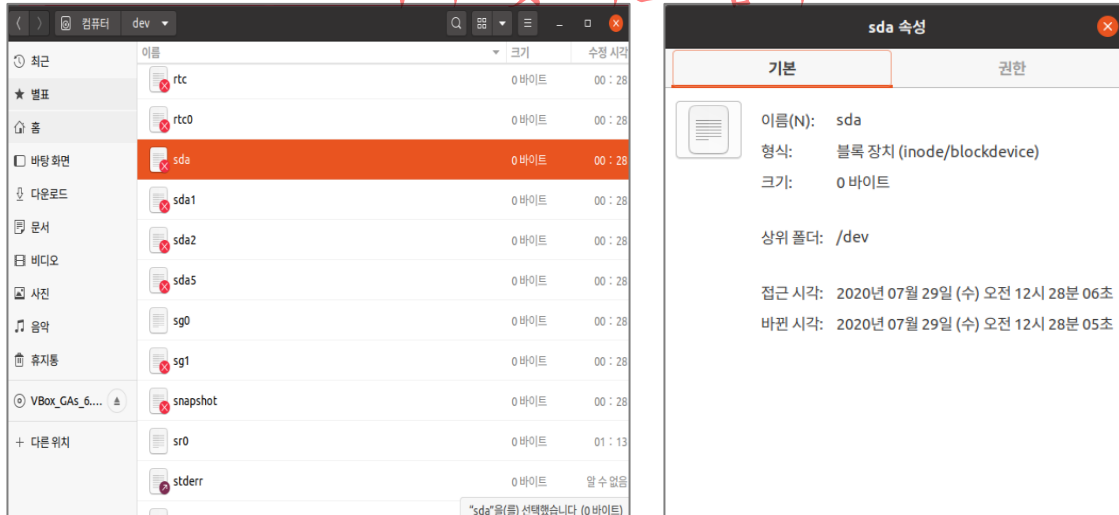


그림 4-6 /dev 디렉터리와 /dev/sda 파일 속성

파이프와 소켓은 프로세스 간 통신에 사용되는 통로이다. 실제 데이터가 파이프나 소켓 파일 내부에 기록되는 것은 아니고, 사용자 프로그램 또는 명령어를 통해서 파일에 데이터를 쓰는 것처럼 하면 상대방에게 데이터가 전송되는 것이고, 파일에서 읽는 것과 같은 방식으로 데이터를 수신할 수 있게 된다. GUI 환경에서는 이들 기능을 사용하기는 어려우며, 추후 명령어와 프로그래밍을 다룰 때 상세한 사항을 살펴 볼 것이다.

4.1.4 하드 디스크 구조

앞에서 장치 파일의 예로 `/dev/sda`를 살펴 보았다. 각 장치 파일은 정해진 명명 방식을 따르는데 `sd`는 SCSI 또는 SATA 방식의 하드디스크를 의미하며, 각 물리적 디스크마다 `a`, `b`, `c` 등의 문자를 추가한다. 또한 물리적 디스크는 논리적으로 분할하여 사용할 수 있는데 이를 파티션이라 하고 파티션에는 번호를 붙여 `sda1`, `sda2`, ... 등과 같이 명명한다. 이러한 하드디스크에 대한 정보를 보려면 **프로그램 표시** → **유틸리티** → **디스크** 아이콘을 선택해 실행하면 된다(프로그래밍: `gnome-disks`).

그림 4-7의 예는 가상 머신에 단일 파티션으로 설치한 경우이기 때문에 매우 단순한 하드 디스크 구조를 갖고 있다. 위쪽은 물리적인 하드디스크에 대한 정보를 나타내는 것으로 모델, 크기, 파티션 만들기, 시리얼 번호, 평가 등을 볼 수 있다. 디스크에 대한 작업은 그림에서 보는 바와 같이 우측 상단 메뉴에 나열되어 있다. 포맷이나 벤치마크, 설정 등의 작업을 할 수 있다.

모든 저작권은 도서출판 흥릉 및
윤현주에게 귀속되어 있음.
복사 및 배포 금지

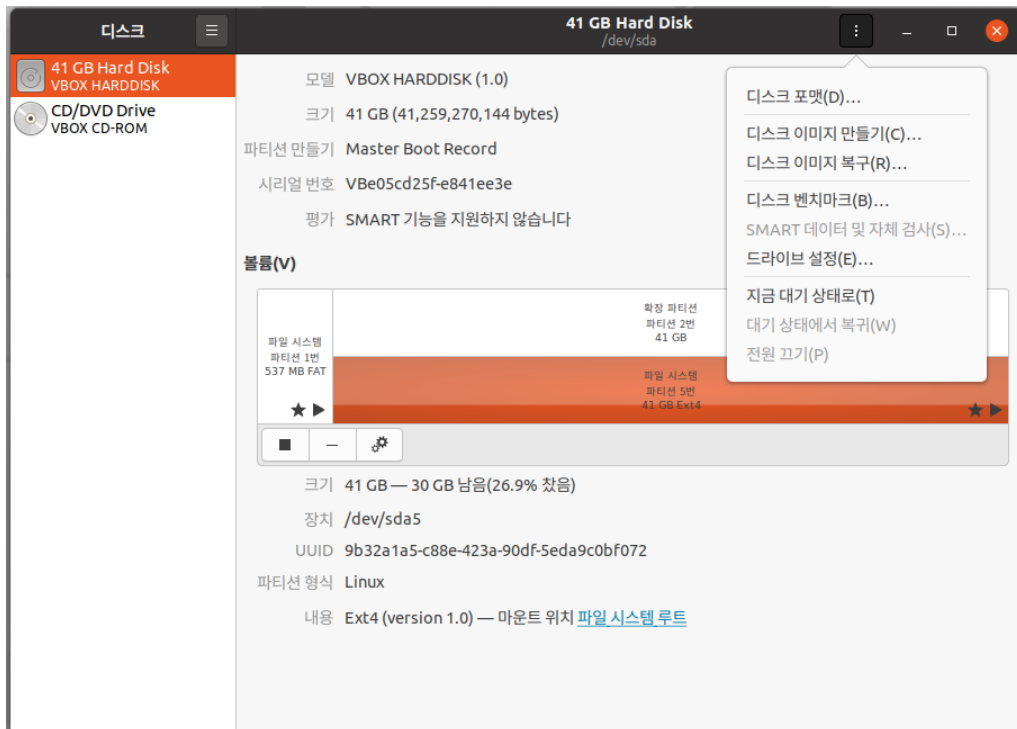


그림 4-7 디스크 도구

아래 **볼륨** 영역은 하드디스크를 논리적으로 분할한 파티션 정보를 나타낸 것이다. 설치할 때 아무런 분할 없이 통으로 설치했지만 자동으로 부트 영역(왼쪽의 파티션 1번)과 리눅스 영역으로 구분되어 있음을 볼 수 있다. 여기의 **장치** 정보를 보면 `/dev/sda5`로 지정되어 있음을 볼 수 있다.

결들임: 파티션의 종류

하드 디스크가 커질수록 관리하기가 어렵기 때문에 용도에 맞추어 분할해서 사용하기 위한 개념이 **파티션**이다. 파티션의 구조와 종류는 물리 수준에 가까운 지점에서 하드 디스크 관리 구조와 맞도록 설정되어 있다. 오래된 MBR(Master Boot Record) 방식에서는 최대 32bit 주소 영역에 맞춘 제약이 몇 가지 존재한다. 먼저 하나의 파티션 크기가 2TiB를 넘을 수 없다. 주(primary) 파티션은 부팅 가능한 영역으로 하나의 하드 디스크에 최대 4개(1번~4번)까지 생성 가능하다. 단순 데이터를 저장하는 용도로 주 파티션을 사용할 필요는 없으므로 논리(logical) 파티션을 생성하게 되는데, 논리 파티션 전에 주 파티션 하나를 확장(extended) 파티션으로 만들고 그 안에 최대 12개까지의 논리 파티션을 만들 수 있다. 논리 파티션은 5번부터 시작한다. 이러한 파티션 정보는 하드 디스크 첫 번째 섹터에 저장되어 부팅 과정에 사용된다. 64-bit 주소 영역을 활용할 수 있는 최근의 GPT(GUID Partition Table) 방식으로는 파티션 종류와 개수, 크기에 제한이 없다.

모든 저작권은 도서출판 흥릉 및 윤현주에게 귀속되어 있음. 복사 및 배포 금지

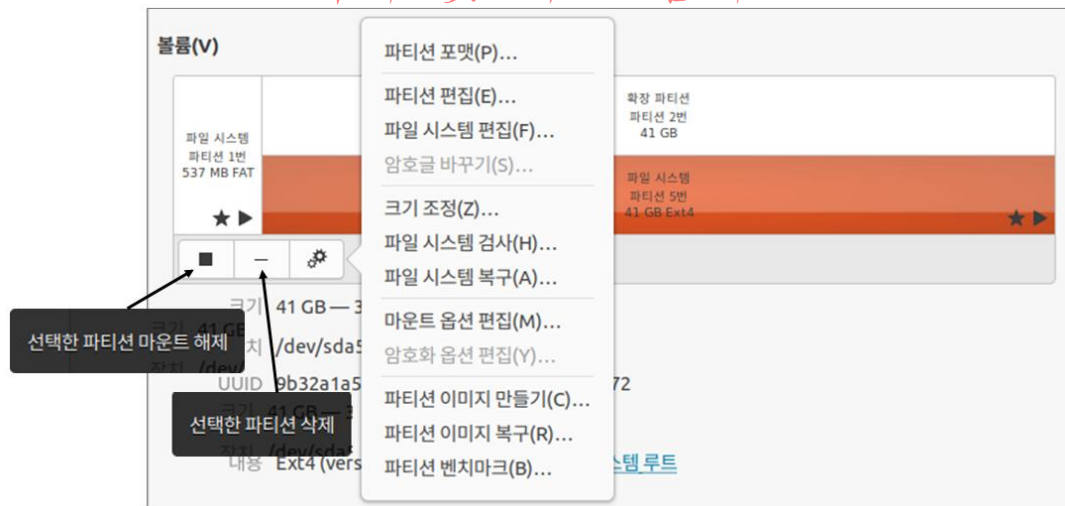


그림 4-8 디스크 도구 - 파티션 관리 메뉴

파티션에 대한 작업은 좀 더 다양하다. 특정 파티션을 선택하고 그 파티션에 대한 작업을 수행할 수 있는데(그림 4-8) 일시적으로 사용을 중지하고 운영 체제에서 분리하는 마운트 해제¹, 논리 파티션 삭제, 기타 크기 조정이나 검사 등의 작업이 가능하다. 포맷이나 삭제 등의 작업은 저장된 데이터를 모두 손실할 위험이 있으므로 예정 없이 실수로 실행하지 않도록 주의한다.

그림 4-9는 1개의 SSD와 2개의 하드 디스크, USB로 연결된 1개의 외장 하드 디스크가 설치된 시스템의 파티션 형태를 보여 준다. 첫 번째 SSD는 GUID-EFI 방식으로 만들어져 총 7개 파티션이 있으며 3번이 Windows C: 드라이브, 7번이 리눅스 루트(/) 파티션이다. 이 시스템의 리눅스는 루트 파티션과 /home 파티션을 분리해 두어 /home 파티션은 두 번째 나타나 있는 하드 디스크의 5번 파티션을 차지하고 있다. 장치 정보를 보면 /dev/sdb5로 지정되어 있는 것을 볼 수 있다. 리눅스 가상 파일 시스템의 특징은 파티션이 나누어져 있어도 하나의 루트만 존재하는 한 개의 트리로 전체 파일 시스템을 표현한다는 것이

다² 이 시스템에서 루트 파티션은 전체 트리에서 /home부터 시작하는 부분 트리를 제외

파일 시스템 구조 : 논리적 저장 단위 -> 물리적 보조 저장 장치에 매핑 알고리즘

리눅스 계층적 파일 시스템은 virtual 계층, logical 계층, 장치제어계층이 있음

unix, linux파일 시스템은 트리구조-> file, directory로 구성 -> directory는 목적이 저장된 파일임 -> folder와의 차이?

경로 = path -> 절대경로(유일무이한 이름의 역할), 상대경로(현재 위치로 부터 경로), 특별한경로이름(/ . .. ~)

홈 디렉토리

심볼릭 링크 = 바로가기,

하드링크 = 절대경로 하나 더 만드는 거임 함부로 사용 L

pwd는 절대경로

ls 옵션

-a 모두나열,

-l 상세정보,

-F 끝부분에 항목 유형을 표시함

-R 현재 디렉토리뿐 아니라 그 안의 모든 하위 폴더까지 출력.

-d 디렉토리 자체만 표시하고, 그 안의 내용을 출력하지 않습니다.

-s 각 파일 앞에 크기(블록 수)가 붙어서 출력.

-l 여러 파일이 세로로 나열됨.

ls -ls했을 때 해석하는법

예시

8-rw-r--r-- 1 user group 3456 Sep 23 14:00 file1.txt

첫 번째 숫자 (예: 8, 4, 12)

-> 파일이 차지하는 디스크 블록 수 (보통 1블록 = 512바이트 또는 1KB, 시스템에 따라 다름)

권한 (-rw-r--r--)

-> 파일 접근 권한 (읽기/쓰기/실행)

링크 수 (1)

-> 해당 파일을 가리키는 하드링크 개수

소유자 (user) / 그룹 (group)

-> 파일의 소유자와 그룹

크기 (3456)

-> 파일의 실제 크기(바이트 단위)

수정 시간 (Sep 23 14:00)

-> 마지막 수정된 날짜와 시간

파일 이름 (file1.txt)

-> 파일 또는 디렉토리 이름